

Partition-Tolerant Federation Across Emulated ISP Islands

Anonymous submission

Abstract—An Internet shutdown is rarely a clean power switch. Transit to the outside world can be withdrawn while domestic networks keep carrying packets among themselves, leaving reachable fragments that recent measurement work calls connectivity islands. We ask a deliberately narrow question: when some domestic IP paths survive, can independently operated forum servers keep accepting signed posts and keep reconciling them across whatever graph remains? Our prototype combines four mechanisms that are individually familiar and, to our knowledge, have not been composed for this operating regime: canonical protobuf envelopes admitted as the exact bytes their author signed, operator receipts bound to those bytes, caller-initiated synchronisation that survives asymmetric reachability, and an explicitly trusted dual-homed relay between islands. On a four-node testbed whose shared exchange segment is withdrawn mid-run, a dependent certificate, community, and post crossed the severed boundary in 0.5, 1.2, and 2.1 seconds—within a factor of two of the same chain measured while the exchange was intact. An eight-node chain delivered 200 of 200 envelopes seven hops with a 4.125s median and a 6.115s 99th percentile, and per-hop cost tracked a queueing model to within 89ms across a fourfold change in the drain interval. TypeScript, Rust, and Python implementations agreed on all 16 canonical vectors. A signature-invalid flood produced no database writes, though a rate limiter shed 90% of it before the pipeline was reached, which we report rather than headline. Against a median gzipped ActivityPub activity our envelopes are 4.4–6.9× smaller, at the cost of Fediverse interoperability and JSON-LD extensibility. These results establish partition tolerance under surviving IP paths on a single-host testbed. They establish nothing about total blackouts, real inter-provider deployment, or resourced denial of service.

Index Terms—federated systems, Internet shutdowns, network partitions, signed objects, store-and-forward

I. INTRODUCTION

Internet shutdown doesn't necessarily mean a complete loss of connectivity. The disruption can occur in several forms like reduction in bandwidth, blocking destinations, filtering protocols, and loss of international routes. Despite these restrictions, domestic networks can still communicate with each other [1], [2]. Baltra et al. described the groups that lost the core internet connection but still can communicate locally as "islands" [3]. They also described networks that still have limited connection with the internet as "peninsulas" [3]. They have explained how these disrupted situations can be identified and developed measurements for them. Our work focuses on developing a system which can continue working in these crucial situations. Our system tries to answer a specific question: if the international routes are cut but domestic IP connection remains connected and a forum server

works independently, whether they can receive signed posts and synchronize those posts across the network?

Other existing systems focus on a related problem which is not exactly similar to ours. Circumvention systems are designed to reach a blocked region. Telex depends on cooperation from network infrastructure to help a blocked user to reach outside the network. On the other hand, Snowflakes does this through proxy servers [4], [5]. In their cases, all of them have to reach either the network infrastructure or the proxy server. But in case of internet cut, they can reach neither them when all external paths are unavailable. Anix, Rangzen, Moby, and Twimight tried to construct a device-to-device or mobile mesh network for a situation where general internet access is not available [6]–[9]. They have used technologies such as Bluetooth or Wi-Fi, where messages pass from one device to another nearby device and the device relay the message farther after receiving it. Though they can continue to messaging while complete blackout, message delivery may take huge time due to network congestion, mobility, intermittent device etc. Also, they might waste additional power and resources of the device. So far we've studied, none of them have studied if independent servers can continue to check, store and relay messages maintaining a strong local network. Our research addresses this gap and tries to find a solution.

Our designed system follows a principal rule: author signs a fixed sequence of bytes, receiving server verifies it without changing any bytes. While forwarding an object through a relay, it can never be repaired, normalized or re-encoded. When a server accepts an object, it then issues a receipt which is cryptographically bound to object's content id and acceptance proof. It not only provides the object's content ID, but also proves that the server accepted the canonical content identified by that ID. Synchronization starts from the caller server, so the server which can't receive incoming connections, can also publish objects and restore missed objects from other servers. If two islands can't communicate directly, they pass messages through a trusted bridge server which is connected to both. The bridge server can verify, save and relay objects. Just like other servers, it does an admission check of the objects.

This paper makes mainly 4 contributions:

- 1) Server follows same byte preserving admission process for both locally saved object and object found from other servers. So, a protocol-compliant relay does not change a malformed object into a valid one.
- 2) It supports caller-initiated delivery, streaming and back-fill. So, even if a server can't accept an incoming

connection, can still publish and recover objects by establishing an outbound connection to known peer.

- 3) It defines a quota bound relay policy. Relay verifies the object again, checks quota. Most importantly, it doesn't send the object back to the server again from which it has received it. A trusted peer may get larger quota and access to additional traffic classes while sending or receiving messages. But it doesn't mean that a post by truster peer is automatically valid.
- 4) It reports a result from a testbed where connection between islands were disconnected while experiment. The experiment also includes cross-language byte agreement tests, performance change with respect to increasing hop, flood of invalid signatures and a comparison with ActivityPub.

Our claim is limited to partition tolerance only when some IP paths remain between servers directly or through a bridge. This proposed design cannot create a new route during complete blackout. This cannot force a server to accept a valid post or hide any visible relay from its hosting network operator.

II. BACKGROUND AND RESEARCH GAP

A. Federated and Signed System

ActivityPub directly delivers social activities to the server's inbox [10]. AT protocol supports federation and repository replication [11], [12]. Nostr and Secure Scuttlebutt replicates object signed by the author [13], [14]. They generally assume that some relevant server, relay, or peer endpoint remains reachable. But none of them has evaluated exact-byte admission, operator receipts, restoration through outbound connection and relay policy for connection between two network islands.

B. Communication During Blackout

Anix, Rangen, Moby and Twimlight establishes a network connecting nearby devices [6]–[9]. Dolphin uses cellular voice channel when normal packet is blocked [15]. These systems work without internal server connection. But they sacrifice bandwidth, devices extra power consumption and network capacity. Our system chooses the opposite trade-off: when the servers are connected, it establishes connection between them, but can't work when all IP paths are totally disconnected. Those systems don't compete with one another, rather they complement each other. So, this paper doesn't compare their latency, because they are designed for different workloads.

C. Accountability:

Append-only logs, signed tree heads and inclusion proofs has been created to audit certain kind of harmful behavior father by certificate transparency [16], [17]. PeerReview and CoSi explores several stronger forms of accountability by using witnesses and cosigning [18], [19]. Our receipt has been kept comparatively weaker intentionally. It only proves that an operator has acknowledged a specific content ID at a stated time. It doesn't prove that operator will receive valid object, will forward object faster or object will not be rejected silently.

D. Study gap and design objective

There is a difference currently present two fields. Normally routed federation assumes that server is always directly connected to the network. On the other hand, device to device assumes that network is totally disrupted. We've studied an intermediate situation between them where independent servers are working and there exists internal paths while the international routes are completely disconnected. In this situation, our goal is to admit each signed object once per server, keep its signed bytes unchanged, and allow every reachable server using the same rules and prerequisite state to reach the same validity decision.

III. SYSTEM DESIGN

A. Architecture

The system is designed around two premises: a forum should keep working as connectivity narrows, and should stress censorship resistance. We have denoted and modeled narrowing as a five-level ladder — L0, ordinary global (or Tor) reach; L1, national partition, where international transit is gone but the domestic exchange still carries traffic; L2, ISP island, where inter-ISP peering itself has failed; L3, bridged islands, where a dual-homed relay reconnects two islands that cannot otherwise reach each other; and L4, total isolation, where no IP path is available.

The ingress is made polymorphic and heterogeneous to match this ladder : HTTP/gRPC is used over whatever IP path survives, a Tor v3 onion service is used when an external Tor path persists (and server / user wants to use that), and an optional Reticulum/LoRa sidecar carrying only small, high-priority messages when IP is unavailable altogether. Bulk traffic is rejected on that path, and it is disabled by default. That sidecar is our only answer to L4, and it is a narrow one: this paper focuses solely the IP-networking path (HTTP for clients, gRPC for federation), and its claims are scoped to L0 through L3.

Edited version:

The canonical signed envelope is invariant across ingress media and relay paths. The transport context and admission context are not. Every node validates the envelope through the same 19-stage admission pipeline. The first 12 stages validate the envelope without persistent writes. If the envelope is accepted, the node commits it, signs a receipt, and places a federation task in a durable outbox. The scheduler drains the outbox by priority class, then by '(destination, source uplink, traffic class)'.

The node must still discriminate how the envelope arrived. Let C_1 denote a client, and let S_1 and S_2 denote independent forum servers. In the illustrative case $C_1 \rightarrow S_1$, the client submits directly to its chosen server. In the illustrative case $C_1 \rightarrow S_1 \rightarrow S_2$, server S_1 relays the same signed envelope to server S_2 . These are examples, not an exhaustive enumeration. The canonical bytes, content identifier, and author signature remain unchanged. What changes is the admission context: ingress kind, supplying peer, direction, source uplink, traffic

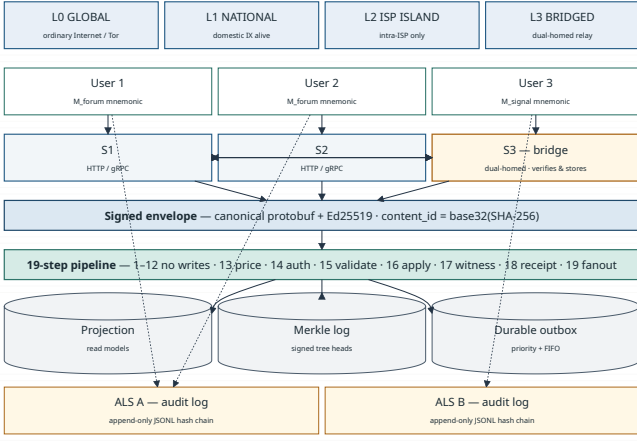


Fig. 1. System architecture. Clients sign canonical envelopes on-device and submit them to forum nodes over HTTPS or gRPC; every node runs the same 19-step admission pipeline, persists accepted envelopes into projections and a Merkle log, and exchanges exact signed bytes with peers via Deliver / StreamActivities / Backfill. Independent audit-log servers (ALS A, ALS B) receive public audit certificates after acceptance. The dual-homed node S3 acts as a bridge under the same admission policy. The reachability scope at the top indicates which paths are usable at a given moment.

class, and quota policy. That context controls deduplication, replay protection, scheduling, and fan-out. It never relaxes signature, identifier, or body validation.

A bridge is not a transparent packet forwarder. It is an ordinary node running the same stack, dual-homed onto two islands. Everything it relays has already been independently validated, stored, and quota-bounded under its own key.

Independent audit-log services (ALSs) provide an external record of server acknowledgement. They operate outside the write path. A server first commits the write and returns a signed receipt. The client builds a public audit certificate, stores it locally, and then forwards it to one or more ALSs. Each ALS independently re-verifies the canonical envelope, content identifier, receipt, signed tree head, and inclusion proof before appending the certificate to its own hash-chained record. If the server later hides or deletes the accepted object, the acknowledgement remains externally verifiable. This supports censorship resistance by limiting a single server’s ability to silently erase evidence of acceptance. Fig. 1 shows the system end to end: the reachability ladder, the on-device signing boundary, the canonical envelope, the shared admission pipeline, each node’s persistence stack, and the independent audit-log services.

B. Signed objects and admission

An envelope carries a domain tag, an object type, a payload, a public key, a nonce, and an expiry. Its identifier is the SHA-256 digest of a specified canonical byte string, and an Ed25519 signature covers that same string [20]. Protobuf does not guarantee a unique serialisation [21], so we constrain a profile on top of it — ascending field order, minimal variants, no unknown fields retained — and add the rule that

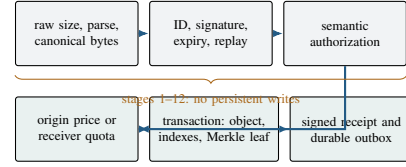


Fig. 2. Admission pipeline. State changes begin only after the cheap byte and validity checks have all succeeded.

actually matters: decoding and re-encoding an object must reproduce the bytes as received, exactly. An object that fails that comparison is rejected outright, never repaired.

Verifying the exact received bytes after a strict decode is established practice in DER, JWS, and Certificate Transparency leaf formats. In this system, the rule protects federation. If a relay decoded a peer’s bytes and re-encoded them before checking the signature, it could normalise a non-canonical submission into the canonical form that was signed. The relay would then verify reconstructed bytes rather than the bytes that arrived. Our decode–re-encode comparison prevents a receiver from repairing or normalising attacker-controlled bytes before verification. As a result, no relay can make a non-canonical or invalid submission acceptable by rewriting it.

Fig. 2 lays out the admission path. Stages 1 through 12 — size, parsing, canonicity, domain separation, identifier derivation, signature, expiry, replay, authorisation — write nothing to persistent storage, so an invalid flood cannot amplify into write load, a property we measure directly in §V. The stages after that charge origin pricing or a receiver quota, commit the raw object with its derived indexes in one transaction, append its hash to a Merkle log, and only then enqueue federation. Exact retries are made idempotent under concurrency by a uniqueness constraint in the database, not by a read-then-write check that a race could defeat.

On acceptance, the server signs a receipt binding the content identifier, the leaf position, the resulting signed tree head, and an inclusion proof. The acknowledgement therefore commits to the exact bytes admitted, not to a label the operator can later change. Peers also exchange signed tree heads containing a tree size, root hash, and timestamp. Two heads of equal size with different roots are an unambiguous conflict. A larger head is recorded as newer, but it is not treated as proven until a consistency proof can be obtained. A smaller head is not immediately treated as a fork, because observations may arrive out of order: if its timestamp is older than the recorded head, it is discarded as stale; otherwise, a smaller size is reported as a possible fork. This is deliberate stale-head hygiene. It removes a class of self-inflicted false positives and catches outright tree shrinkage, but it is not full fork detection. An adversary who rewrites history while keeping the tree size non-decreasing can pass these local checks, and equivocation between two peers cannot be detected by a single local comparison. Detecting those cases requires consistency proofs and independent cross-observer exchange, which the network partitions we target are precisely likely to obstruct.

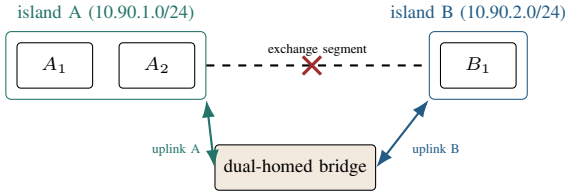


Fig. 3. Route-cut topology. Four application nodes span two isolated network segments joined by a shared exchange, which the experiment withdraws mid-run. The bridge is an application server, not an IP forwarder.

C. Partition-aware federation

Three caller-initiated RPCs cover ordinary delivery and recovery. **DELIVER** pushes a batch of accepted envelopes to a peer. **STREAMACTIVITIES** resumes a live stream from a durable cursor. **BACKFILL** requests objects that the node knows it missed, using a resumable log position. Because all three operations are initiated by the caller, a node does not need to accept inbound federation connections. A server behind carrier-grade NAT can therefore federate in both directions over connections it opens itself. This solves reachability asymmetry, not discovery: at least one peer address must already be known.

The scheduler keeps an independent durable queue for each destination peer. It drains these queues concurrently under bounded leases, with deadlines, exponential backoff, and idempotency keys. This structure exists because its absence was a real defect, described in §V: a serial drain allowed one unreachable peer to block delivery to every other peer, including the bridge. One caveat belongs here rather than in the limitations section. A delivery deadline bounds how long the drain pass waits on a peer; it does not cancel the underlying transport call. Because the sender holds no cancellation token, a stuck call can continue to hold resources after its deadline, even though it no longer blocks delivery to other peers.

Fig. 3 shows the route-cut topology used in the evaluation. Island A contains two ordinary nodes, island B contains one, and a shared exchange segment stands in for an Internet exchange point. The bridge holds one address on each island network. Once the exchange segment is withdrawn, both islands must already have configured trust in the bridge. That trust determines the rate and priority of the bridge’s traffic; it never determines whether an envelope is valid. The bridge validates and stores every object it relays, enforces quota per uplink pair and traffic class, and never sends an object back to the peer from which it arrived.

Four implementation rules carry most of the replication safety. Peer bytes are passed through unchanged: the transport passes received envelopes as opaque byte strings and does not decode or re-encode them. Trust controls quota and priority, never validity; every peer, including the bridge, still repeats the full admission checks. Duplicate delivery is inevitable when a live stream and a backfill race, and it is absorbed by a database uniqueness constraint rather than by a read-then-write check. Fan-out excludes the peer that supplied an object, avoiding

a redundant round trip that deduplication would eventually terminate, but not for free.

IV. METHODOLOGY

We evaluate five questions: whether independent encoders agree on the canonical form, whether objects cross an enforced route cut, how delivery latency scales with hop count, whether malformed requests produce storage writes, and how the representation compares in size with ActivityPub. All experiments run on one physical host; §VI treats the consequences of that choice at length.

Correctness tests. A shared corpus of 16 positive and adversarial byte vectors is executed independently by the TypeScript, Rust, and Python implementations. Each must agree byte-for-byte on encoding and identifier derivation, and each must reject every negative vector. A further 85 focused tests cover ingress (29), the outbox (3), federation (31), and simulated ISP behaviour (22). These are regression tests written by the system’s authors, not an independent security audit.

Route-cut experiment. Four application nodes, two databases, and two caches are distributed across three isolated virtual network segments, as shown in Fig. 3. Containers supply reproducible layer-2 isolation and let the harness withdraw the only configured inter-island path mid-run; nothing in the design depends on that packaging.

The test proceeds in four phases. First, the gate reads kernel TCP state from `/proc/net/tcp` inside the bridge to confirm which local source address each established federation connection actually left from. Second, it withdraws the exchange segment and verifies that direct probes between islands now fail. Third, it confirms that island B does not already hold the content to be submitted. Fourth, it submits a dependency chain—an author certificate, a community record, and a post that depends on both—and polls for each exact content identifier on the far island. The chain is ordered because a post cannot be rendered until its author certificate and community record are present, so the measured crossing reports all three arrivals rather than only the post event.

Nineteen assertions cover topology, source binding, relay behaviour, quota reservation, and uplink failover. Reported latencies include the application’s own polling interval and are therefore upper bounds.

Hop-scaling chain. Eight nodes are peered into a line, each with its own database, so that a node i hops from the origin is reachable by exactly one path of exactly that length. Hop count is the independent variable. A mesh would destroy this property, since in a mesh every node is one hop away and only fan-out width varies.

Latency is computed by subtracting the `received_at_ms` value each node writes inside the same transaction as its projection. No polling observer sits in the measurement path. This is sound only because all nodes share the host clock, which a real deployment would have to replace with NTP discipline and an error bar. We inject 200 valid envelopes at hop 0 at a paced rate and record arrival at

hop 7. The eight nodes share a single database process with one database each, which introduces contention and inflates the reported figures—the conservative direction in which to be wrong.

Signature-invalid flood. Random bytes are refused within microseconds at the parse stage and measure nothing interesting. The tool therefore lifts a genuine envelope from the node’s own log and mutates a single byte inside the signed region. This invalidates the signature and changes the content identifier, so every request is novel and none can be short-circuited by deduplication. We issue 3,000 such requests at concurrency 16 and record completion rate, rejection stage, the fraction reaching signature verification, and—via the database’s collection-level profiler—the resulting writes. The ordinary rate limiter stays enabled, so this is not a saturation benchmark.

Encoding comparison. We collected 80 real Create/Note activities from the public outboxes of two stock ActivityPub deployments. Because a collection hoists @context to the collection while an activity POSTed to a remote inbox carries its own, measuring the bare outbox item would understate delivery size. The tool therefore refetches a standalone object from the same host and re-attaches exactly those bytes. We compare raw and gzipped activity bytes against equivalent signed envelopes, subtracting authored UTF-8 text so the metric reflects what a protocol designer controls rather than how much a user typed.

The comparison is deliberately generous to ActivityPub. It excludes HTTP framing, TLS, per-follower fan-out, shared dictionaries, and—most significantly—authentication, since our bytes carry a 64-byte signature verifiable at rest while HTTP Signatures are transport level and vanish with the request.

V. RESULTS

A. Correctness and convergence

All three implementations agreed on 16 of 16 canonical vectors and rejected every negative vector as specified. The focused suites additionally exercise invalid signatures, non-canonical byte layouts, replay, missing authorisation dependencies, concurrent duplicate delivery, receipts signed by the wrong key, invalid inclusion proofs, stale tree heads, and equal-size log conflicts. Sixteen vectors plainly do not exhaust protobuf’s edge cases; what they establish is that three independently written implementations can be held to one byte-exact profile, which is the property a relay network depends on.

The route-cut gate passed all 19 assertions. Source-binding verification via kernel TCP state confirmed established federation connections leaving the bridge from both configured addresses, 10.90.1.30 and 10.90.2.30—an assertion no in-process harness can make, because in process there are no sockets to inspect. After direct inter-island reachability was withdrawn, island B received the certificate 0.5 s, the community 1.2 s, and the post 2.1 s after publication, in dependency order, by way of the bridge. The control matters as much as the result:

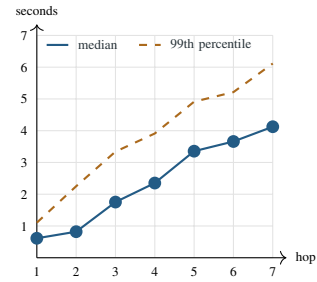


Fig. 4. Measured per-hop latency across the eight-node chain at a 1,000 ms drain interval, from in-transaction timestamps. All 200 envelopes reached hop 7.

the same chain measured with the exchange intact took 1.5, 1.5, and 1.9 s, so the relayed path is not merely functional but within a factor of two of the direct one. These are single preserved runs, not distributions.

That number was earned rather than observed. An earlier build passed 18 of 19, and the crossing it did complete took 407.6 s, with every outbox row still recording zero delivery attempts. The cause was serial per-peer draining with no deadline: one blackholed peer stalled delivery to every other peer in the same pass, including the bridge, whose entire purpose was to route around the partition causing the stall. Draining peers concurrently and bounding each delivery reduced the crossing to 2.1 s, a factor of 195. The defect is invisible to a two-peer harness by construction—it requires three peers, one of which becomes unreachable while the others remain up—and this is our strongest argument for evaluating federation on a deployed multi-peer testbed rather than in process.

B. Latency scaling and admission cost

All 200 envelopes traversed the eight-node chain to hop 7. End-to-end median latency was 4.125 s and the 99th percentile 6.115 s, with the per-hop distribution in Fig. 4. The curve is more informative than its endpoint. Per-hop cost is dominated by the outbox drain interval rather than by the protocol, predicting roughly $drain/2$ plus a fixed term; at a 1,000 ms drain the seven hops cost 589 ms each, implying an 89 ms fixed component, and at 250 ms they cost 196 ms each, implying 71 ms. A residual stable at 71–89 ms across a fourfold change in the queueing parameter is the protocol’s genuine per-hop cost—verify, project, append, re-enqueue—and everything above it is a scheduling choice an operator makes for bandwidth and battery reasons. Contention on the shared database process and the absence of wide-area delay both mean this curve should not be read as a prediction for an Internet deployment.

The signature-invalid flood was rejected at 730 requests per second and produced zero object, index, log, or outbox writes. The negative result carries a control, since “the profiler recorded no writes” is worthless if the profiler was not running: the same profile captured six read operations over the window, establishing that it was live. We report the composition of that traffic rather than the throughput figure, because the

composition is what qualifies the claim. Roughly 2,700 of the 3,000 requests were shed by the per-peer rate limiter, so only about 9% reached signature verification, and the expensive rejection path—canonical parse followed by Ed25519 verify—was therefore exercised at an effective rate near 66 requests per second. The supported conclusion is narrow: for this traffic, the no-write prefix held, and it did so as the second line of defence behind a token bucket that did most of the work. The experiment says nothing about distributed floods across many peer identities, about correctly signed envelopes that are *supposed* to produce writes, or about sustained parser and database pressure.

Table I reports the encoding comparison and the costs the format incurs. Both sampled hosts emit an identical 927-byte `@context` declaration—minimum equal to maximum on each—so it is a fixed per-activity cost of the stock dialect rather than a deployment artefact, and that declaration alone is over four times the size of our entire signed forum post. Compression erases most of the apparent advantage and we lead with the compressed figure for that reason. Gzipped ActivityPub activities ranged 786–1,400 B with a median of 1,074 B, against raw signed envelopes of 155 B, 220 B, and 243 B: a 4.4–6.9 $\times$ ratio. Our envelopes do not compress at all—a 64-byte Ed25519 signature is incompressible and gzip makes all three slightly *larger*—so the honest comparison is gzipped ActivityPub against our raw bytes, not the roughly 20 $\times$ ratio the uncompressed figures would suggest. Size is also the only axis on which we win. The format cannot join the existing Fediverse, requires generated codecs and unfamiliar operational tooling, rejects unknown fields so that schema evolution demands a versioned domain, and gives up the open extensibility that is JSON-LD’s reason for existing.

VI. DISCUSSION AND LIMITATIONS

Testbed validity. Every experiment ran on one physical host using virtual network segments, and the resulting evidence ceiling is the paper’s principal weakness. All nodes share a kernel, clock, CPU, and storage subsystem; the shared clock is what makes cross-node timestamp subtraction valid at all, and it is exactly what a real deployment does not provide. Virtual segments, static addressing, and local name resolution are far simpler than physical routers, BGP reconvergence, carrier-grade NAT, DNS failure, variable MTUs, and radio access, and attaching a dual-homed bridge to two virtual networks is trivial where a real one needs separate links, routing policy, credentials, power, and an operator willing to be identified. The route-cut experiment therefore demonstrates a change in software and routing state, not geographic latency, independent failure domains, or provider behaviour.

Access and subscriber isolation. The design assumes nodes within an island can reach one another or a shared bridge. Wi-Fi client isolation at an access point, or subscriber isolation within an ISP, can block customer-to-customer traffic while leaving every customer able to reach the provider gateway. Where that isolation holds on every surviving path, nodes can neither dial each other nor reach a bridge, federation

fails entirely, and only isolated local service remains. The prototype has no non-IP fallback for this case.

Network adversary. A network operator can enumerate stable endpoints, block addresses or SNI values, fingerprint the RPC traffic, correlate timing, isolate a node, or coerce a bridge operator. Source-address binding selects an uplink; it conceals nothing. One shutdown model defeats us outright and deserves naming rather than omission: protocol whitelisting, in which domestic routing stays intact but deep packet inspection admits only sanctioned protocols. Our federation traffic is a distinctive non-standard protocol on a distinctive port, so under whitelisting it dies on the first day regardless of how sound its signatures are. Carrying federation over ordinary HTTPS, rotating endpoints, and adopting pluggable transports are the obvious responses, and none of them is implemented or evaluated here.

System and application limits. Signatures authenticate keys, not truthfulness or safety, and a legitimate key can publish abuse or an expensive flood. Because admission cost is charged at the origin, an origin operator controls its own secrets and can mint unlimited free proofs for its own users; anti-abuse strength is therefore bounded by the policy of the weakest origin in a peer set, and per-peer quotas are the only backstop. Key theft, metadata exposure, moderation at scale, revocation recovery, storage exhaustion, and coordinated Sybil behaviour all remain open. Transport deadlines bound waiting rather than cancelling work. Consistency proofs for growing trees are specified but not implemented. Most fundamentally, a receipt evidences only what was acknowledged: an operator who declines admission outright leaves no receipt and hence no evidence, so the cheapest form of censorship—silence—is the one this construction cannot document.

Evaluation limits. The route-cut figures come from single preserved runs under one configuration, and the encoding sample is 80 activities of a single type from two deployments. We have no multi-host, wide-area, multi-day, or real-shutdown trials, and did not preserve host load, repeated-run distributions, or confidence intervals. Resource figures were taken on x86 hardware and say nothing about the single-board computers a community deployment would realistically use. Comparisons against proximity-based and voice-channel systems concern different workloads and are not a ranking.

Ethics. All tests used synthetic content on isolated private networks. A real deployment could expose bridge operators and users to legal or physical risk, since a visible relay is by construction legible to the network that hosts it, and should not be attempted in a hostile setting without local threat analysis, informed consent, data minimisation, and community review.

VII. CONCLUSION

Server federation can survive a route cut when domestic IP paths remain. Canonical signed objects, a single admission path for local and federated traffic, caller-initiated recovery, and an explicitly trusted validating relay carried a dependent post across a severed exchange in 2.1 s, within a factor of two of the intact path, and the defect that stood between

TABLE I

MEASURED REPRESENTATION SIZE AND DESIGN TRADE-OFFS AGAINST ACTIVITYPUB. THE BYTE SAMPLE IS 80 REAL CREATE/NOTE ACTIVITIES FROM TWO STOCK DEPLOYMENTS. RATIOS COMPARE GZIPPED ACTIVITYPUB AGAINST RAW SIGNED ENVELOPES, WHICH DO NOT COMPRESS.

Dimension	This work	ActivityPub comparison	Cost or boundary
Measured bytes	Raw signed envelopes: check-in 155 B, forum post 220 B, broadcast 243 B; gzip enlarges all three	Delivered JSON-LD median 4,216 B, gzipped median 1,074 B (range 786–1,400 B); @context alone a fixed 927 B	Ratio 4.4–6.9×; representation only—HTTP, TLS, fan-out, shared dictionaries, and AP authentication all excluded in AP’s favour
Interoperability	Custom protobuf domains and RPCs	Standard actor, object, and inbox model	No Fediverse compatibility and no existing application ecosystem
Evolution	Unknown fields rejected; versioned profiles required	Flexible JSON-LD vocabulary and open extension	Harder rolling upgrades and more cross-language test surface
Operations	Generated codecs and canonical-byte tooling required	Text inspectable with ordinary web tools	Higher debugging and tooling burden on operators
Integrity profile	One exact byte string per object, content ID, author signature, and receipt	Authentication depends on deployment profile; HTTP Signatures do not survive the request	Unambiguous byte identity, but a new security profile to maintain and audit

407.6 s and that figure was visible only once three peers were deployed with one of them unreachable. Cross-language byte agreement and a hop-scaling chain whose per-hop cost tracks a queueing model to within 89 ms support the mechanism from two further directions. The compact representation is 4.4–6.9× smaller than gzipped ActivityPub and forfeits interoperability, extensibility, and mature tooling to get there. The work we consider most urgent is multi-host and wide-area evaluation, consistency proofs for growing logs, cancellation-aware transport, floods of validly signed traffic, redundant independent bridges, transport camouflage against protocol whitelisting, and a non-IP fallback for subscriber isolation and total blackouts.

REFERENCES

- [1] Z. S. Bischof, K. Pitcher, E. Carisimo, A. Meng, R. B. Nunes, R. Padmanabhan, M. E. Roberts, A. C. Snoeren, and A. Dainotti, “Destination unreachable: Characterizing internet outages and shutdowns,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023.
- [2] T. I. Raso, S. Afrin, M. A. Chowdhury, M. Xynou, and E. Yachmeneva, “The longest silence: Internet shutdowns during Bangladesh’s 2024 uprising,” Digitally Right and Open Observatory of Network Interference, 2025, <https://ooni.org/post/2025-bangladesh-report/>.
- [3] G. Baltra, T. Saluja, Y. Pradkin, and J. Heidemann, “Reasoning about internet connectivity,” 2024.
- [4] E. Wustrow, S. Wolchok, I. Goldberg, and J. A. Halderman, “Telex: Anticensorship in the network infrastructure,” in *Proceedings of the 20th USENIX Security Symposium*, 2011.
- [5] C. Bocovich, A. Breault, D. Fifield, Serene, and X. Wang, “Snowflake, a censorship circumvention system using temporary WebRTC proxies,” in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [6] S. Kamali and D. Barradas, “Anix: Anonymous blackout-resistant microblogging with message endorsing,” in *2025 IEEE Symposium on Security and Privacy*, 2025, pp. 1381–1399.
- [7] A. Lerner, G. Fanti, Y. Ben-David, J. Garcia, P. Schmitt, and B. Raghavan, “Rangzen: Anonymously getting the word out in a blackout,” 2016.
- [8] A. Pradeep, H. Javaid, R. Williams, A. Rault, D. Choffnes, S. Le Blond, and B. Ford, “Moby: A blackout-resistant anonymity network for mobile devices,” *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 3, pp. 247–267, 2022.
- [9] T. Hossmann, P. Carta, D. Schatzmann, F. Legendre, P. Gunningberg, and C. Rohner, “Twitter in disaster mode: Security architecture,” in *Proceedings of the First ACM Workshop on Privacy and Security in Mobile Systems*, 2011.
- [10] C. L. Webber, J. Tallon, E. Shepherd, A. Guy, and E. Prodromou, “ActivityPub,” W3C Recommendation, 2018, <https://www.w3.org/TR/activitypub/>.
- [11] The Matrix.org Foundation, “Matrix specification, version 1.11,” 2024, <https://spec.matrix.org/v1.11/>.
- [12] Bluesky Social, “AT Protocol repository specification,” 2026, <https://atproto.com/specs/repository>.
- [13] fiatjaf, “NIP-01: Basic protocol flow description,” 2023, <https://github.com/nostr-protocol/nips/blob/master/01.md>.
- [14] D. Tarr, E. Lavoie, A. Meyer, and C. Tschudin, “Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications,” in *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*, 2019, pp. 1–11.
- [15] P. K. Sharma, R. Sharma, K. Singh, M. Maity, and S. Chakravarty, “Dolphin: A cellular voice based internet shutdown resistance system,” *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 1, pp. 589–607, 2023.
- [16] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” Internet Engineering Task Force, Tech. Rep. RFC 6962, 2013.
- [17] B. Laurie, E. Messeri, and R. Stradling, “Certificate transparency version 2.0,” Internet Engineering Task Force, Tech. Rep. RFC 9162, 2021.

- [18] A. Haeberlen, P. Kouznetsov, and P. Druschel, "PeerReview: Practical accountability for distributed systems," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007, pp. 175–188.
- [19] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities honest or bust with decentralized witness cosigning," in *2016 IEEE Symposium on Security and Privacy*, 2016, pp. 526–545.
- [20] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [21] Google, "Protobuf serialization is not canonical," 2024, <https://protobuf.dev/programming-guides/serialization-not-canonical/>.